

ACTIVIDAD 2: Front-End para el Sistema de Gestión de Alquiler de Vehículos

Kevin Andres Jimenez Casares

Facultad de Ingeniería y Tecnología, Fundación Universitaria Internacional de La Rioja

Desarrollo de aplicaciones web

Ernesto Solano Hernandez

25 de mayo del 2026

Índice

Índice	2
Introducción	3
Componentes de React	4
Reutilizables	4
StatusBadge	4
MessageAlert	4
Header	4
LoadingSpinner	4
VehicleForm	4
VehicleTable	4
VehicleRow	4
VehicleDetailCard	4
De página	5
WelcomeScreen	5
Dashboard	5
CreateVehicle	5
EditVehicle	5
App	5
Hooks	5
Custom hook	6
React Router	8
Navegación entre rutas	8
Flujos de navegación	9
Panel principal	9
Panel de administración	9
Listar vehículos	10
Crear vehículo	10
Editar vehículo	10
Eliminar vehículo	10
Panel de alquiler	13

Edición de alquiler.....	13
Consumo de APIs REST.....	15
Arquitectura.....	15
Funcionamiento interno.....	16
Flujo de alquiler.....	16
Conclusiones.....	17
Glosario.....	17
Bibliografía.....	19

Introducción

En este trabajo se desarrolló el Front-End de un sistema de alquiler de vehículos utilizando React, TypeScript y Vite. El objetivo principal fue crear una aplicación que permitiera visualizar vehículos, administrar su información y realizar procesos básicos de alquiler de una forma más organizada y sencilla.

Durante el desarrollo se trabajó con componentes, hooks y rutas para entender mejor cómo se estructura una aplicación moderna en React. Aunque todavía hay conceptos que son nuevos y algo complejos, se logró comprender la importancia de dividir el proyecto en partes pequeñas reutilizables para facilitar el mantenimiento y el orden del código. También se entendió cómo conectar el Front-End con servicios del Back-End mediante APIs REST.

Además, el proyecto permitió practicar temas importantes como navegación entre páginas, manejo de estados y actualización dinámica de información sin necesidad de recargar la aplicación. En general, esta actividad ayudó a tener una idea más clara de cómo funciona el desarrollo web moderno.

Componentes de React

Es una función de JavaScript/TypeScript que retorna JSX (una descripción de lo que se debe renderizar en la UI). Cada componente encapsula su propia estructura, estilo y lógica, y puede recibir datos externos via props. Se componen unos dentro de otros como piezas de Lego para construir la interfaz completa. La aplicación cuenta con 13 componentes funcionales, organizados en tres categorías:

Reutilizables

StatusBadge

Muestra una etiqueta visual (pill) con el estado del vehículo: "Disponible" (verde) o "No disponible" (rojo).

MessageAlert

Renderiza un mensaje emergente y permite cerrarlo al hacer clic. Oculta si no hay mensaje.

Header

Barra de navegación superior con botones para acceder al dashboard, crear y editar vehículos. Se oculta automáticamente en la pantalla de bienvenida.

LoadingSpinner

Animación de carga simple (círculo giratorio) para indicar procesos en segundo plano.

VehicleForm

Formulario reutilizable con campos Marca, Modelo y Estado. Acepta datos iniciales (para edición) y expone callbacks onSubmit y onCancel.

VehicleTable

Tabla completa que lista los vehículos con columnas ID, Marca, Modelo, Estado, Creado, Devolución y Acciones. Integra VehicleRow para cada fila y un botón de recarga.

VehicleRow

Fila individual de la tabla con botones de acción: Ver detalle, Alquilar, Editar y Eliminar. Usa StatusBadge para mostrar el estado.

VehicleDetailCard

Panel de detalle que muestra toda la información de un vehículo seleccionado en una grilla.

De página

WelcomeScreen

Pantalla de inicio con diseño minimalista. Invita al usuario a acceder al dashboard.

Dashboard

Página principal que orquesta la tabla de vehículos, panel de detalle y mensajes.

CreateVehicle

Página que envuelve VehicleForm para la creación de un nuevo vehículo.

EditVehicle

Página que envuelve VehicleForm con datos iniciales para editar un vehículo existente.

App

Componente raíz que configura BrowserRouter con las 4 rutas (/, /dashboard, /create, /edit) y renderiza Header.

Hooks

Son funciones especiales de React que permiten "enganchar" funcionalidades como estado, efectos secundarios o memorización dentro de un componente funcional. Comienzan con use (ej: useState, useEffect). Resuelven la limitación de que los componentes funcionales, por sí solos, no tienen estado ni ciclo de vida.

- useState → agrega estado local que persiste entre renderizados
- useEffect → ejecuta código después del renderizado (llamadas API, suscripciones)
- Custom hooks → funciones que agrupan hooks nativos para abstraer lógica repetitiva (ej: useVehicles)

Hook: useState

Donde se usa: Todos los componentes

Proposito: Maneja estado local: lista de vehículos, formulario, mensajes, loading, etc.

Hook: useEffect

Donde se usa: Dashboard

Proposito: Dispara la carga inicial de vehículos al montar el componente.

Hook: useCallback

Donde se usa: useVehicles

Proposito: Memoriza funciones asíncronas para evitar re-creaciones innecesarias en renders.

Hook: useNavigate

Donde se usa: Header, WelcomeScreen, Dashboard, CreateVehicle, EditVehicle

Proposito: Navega entre rutas del sistema.

Hook: useLocation

Donde se usa: Header

Proposito: Detecta la ruta actual para ocultar el Header en la pantalla de bienvenida.

Custom hook

useVehicles

Archivo: src/hooks/useVehicles.ts

Abstrae toda la lógica de negocio relacionada con vehículos en un solo lugar, evitando que cada componente tenga que repetir llamadas API, gestión de estados y manejo de errores.

Estado que expone:

vehicles — lista completa de vehículos

selectedVehicle — vehículo seleccionado para ver detalle

message — mensaje informativo o de error

loading — indicador de carga

Métodos que expone:

loadVehicles() — obtiene la lista desde la API

viewDetails(id) — consulta un vehículo por ID

rentVehicle(vehicle) — registra un alquiler y actualiza el estado a "no disponible"

removeVehicle(id) — elimina un vehículo de la vista local

clearMessage() — limpia el mensaje actual

Uso en componentes:

```
const { vehicles, loading, loadVehicles, rentVehicle } = useVehicles()
```

Se consume exclusivamente en el componente Dashboard, pero está diseñado para ser reutilizable en cualquier otro componente que necesite interactuar con la flota de vehículos.

React Router

Se implementó react-router-dom para manejar la navegación entre páginas como una SPA (Single Page Application) — las rutas cambian sin recargar el navegador.

Configuración (App.tsx)

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<WelcomeScreen />} />
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/create" element={<CreateVehicle />} />
    <Route path="/edit" element={<EditVehicle />} />
  </Routes>
</BrowserRouter>
```

Navegación entre rutas

Se usa el hook useNavigate para cambiar de página desde cualquier componente:

```
const navigate = useNavigate()

navigate('/dashboard') // ir al listado

navigate('/create')    // ir al formulario de creación
```

Nombre ruta	Ruta path	Componente	Funcionalidad
Página principal	/	WelcomeScreen	Vista general del sistema con bienvenida y acceso al dashboard
Listado de vehículos	/admin	Dashboard	Tabla con todos los vehículos, su estado, y acciones (editar, eliminar)
Alquiler de vehículo	(en /dashboard)	VehicleDetailCard	Panel que se despliega al hacer clic en "Ver" sobre un vehículo en la tabla, mostrando toda su información
Gestión administrativa	/create y /edit	CreateVehicle y EditVehicle	Formularios para registrar un nuevo vehículo o actualizar su estado (usan el componente reutilizable VehicleForm)

Flujos de navegación

Panel principal

El panel principal se compone de dos botones, el primero es una redirección al módulo de realizar los alquileres de los vehículos y el otro es directo a la administración de vehículos.

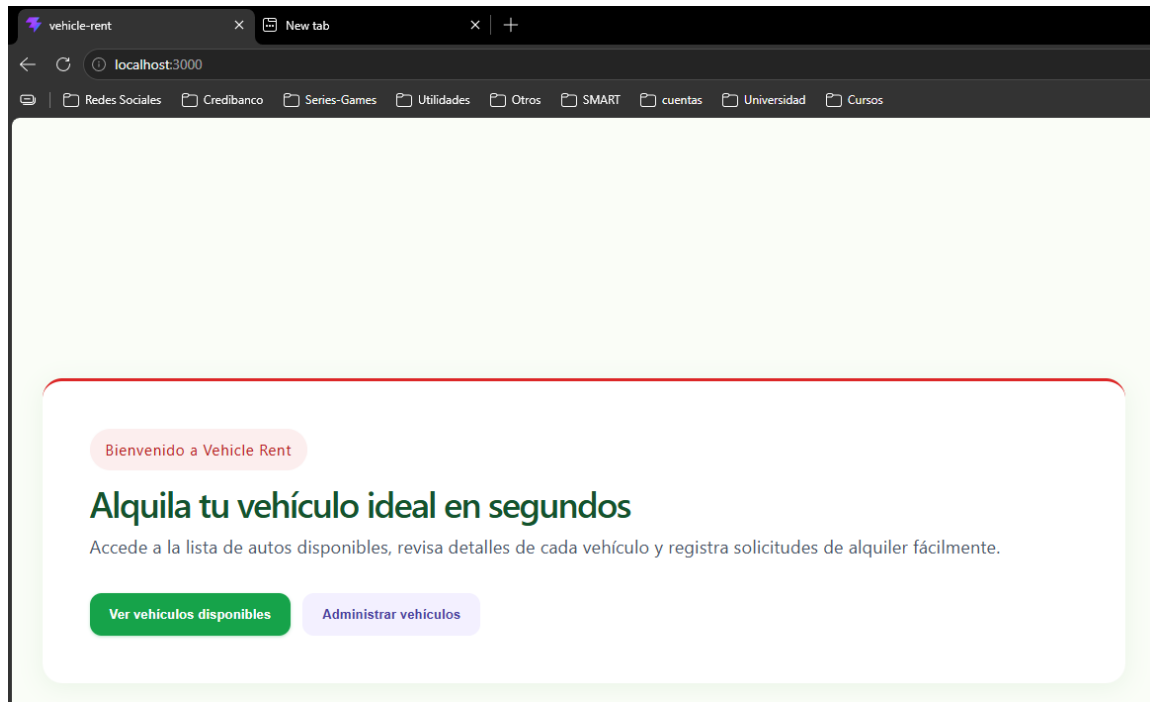


Imagen 1. Panel principal de la aplicación de alquiler de vehículos

Panel de administración

El panel de administración de vehículos busca ser un modulo sencillo donde el usuario cuente con las opciones especificas de administración como crear vehículo, editar vehículo y eliminar vehículo, estas opciones estarán disponibles y permite al usuario gestionar todo sin navegar.

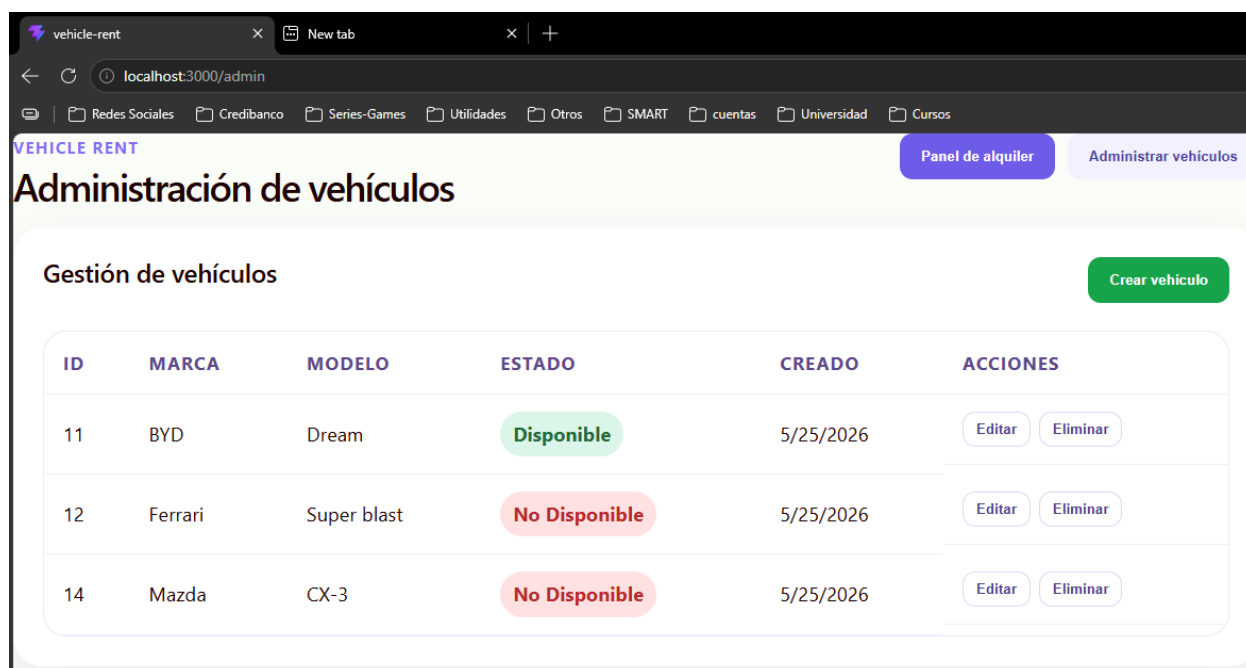


Imagen 2. Panel principal de vehículos

Listar vehículos

Cuando el usuario está en el panel de administración podrá visualizar todos los vehículos creados con los campos id, marca, modelo, estado y la fecha de creación. A su vez va a contar las opciones de editar o eliminar correspondiente a cada registró. **Ver imagen 2**

Crear vehículo

Al seleccionar la opción de crear vehículo, se abrirá un formulario en la misma pantalla con las opciones de marca, modelo y el estado entre disponible y no disponible, una vez se acepte la creación desde los servicios del back se realiza la creación asignando la fecha. **Ver imagen 3**

Editar vehículo

Al seleccionar un vehículo y escoger editar, se abrirá el mismo formulario de creación de vehículo permitido editar los valores de marca, modelo y estado disponible. **Ver imagen 4**

Eliminar vehículo

Cuando el usuario quiere realizar la eliminación de un vehículo, solo debe darle clic al botón de eliminar correspondiente a la fila del vehículo, esto abrirá una ventana en donde podrá confirmar la eliminación del vehículo. **Ver imagen 5 e imagen 6**

Crear vehículo

Marca:
 Modelo:
 Estado:

ID	MARCA	MODELO	ESTADO	CREADO	ACCIONES
2	CHEVROLET	SEDAN	Disponible	5/11/2026	Editar Eliminar
11	BYD	Dream	Disponible	5/25/2026	Editar Eliminar
12	Ferrari	Super blast	No Disponible	5/25/2026	Editar Eliminar
13	Mazda	R1	Disponible	5/25/2026	Editar Eliminar

Vehículo creado correctamente.

Imagen 3. Creación de vehículos en el panel principal

Editar vehículo

Marca:
 Modelo:
 Estado:

ID	MARCA	MODELO	ESTADO	CREADO	ACCIONES
2	CHEVROLET	SEDAN	Disponible	5/11/2026	Editar Eliminar
11	BYD	Dream	Disponible	5/25/2026	Editar Eliminar
12	Ferrari	Super blast	No Disponible	5/25/2026	Editar Eliminar
14	Mazda	CX-3	No Disponible	5/25/2026	Editar Eliminar

Imagen 4. Edición de vehículo desde el panel principal

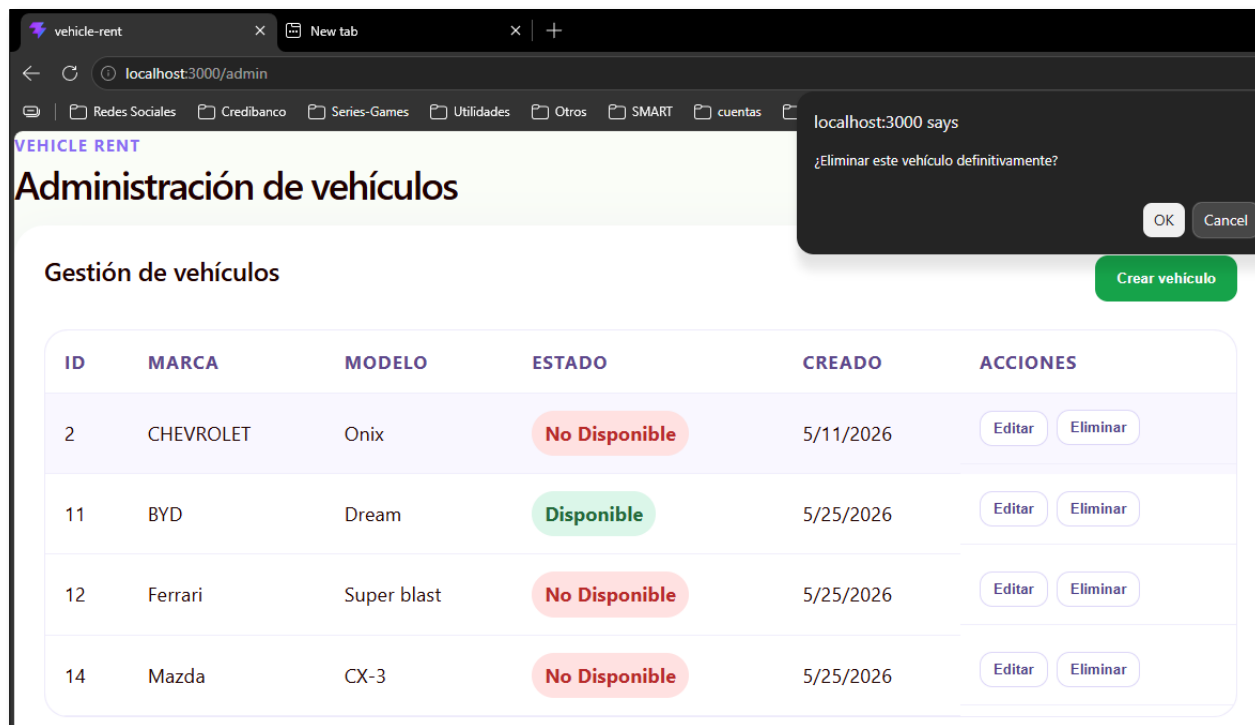


Imagen 5. Opción de eliminación de con apertura de ventana de confirmación

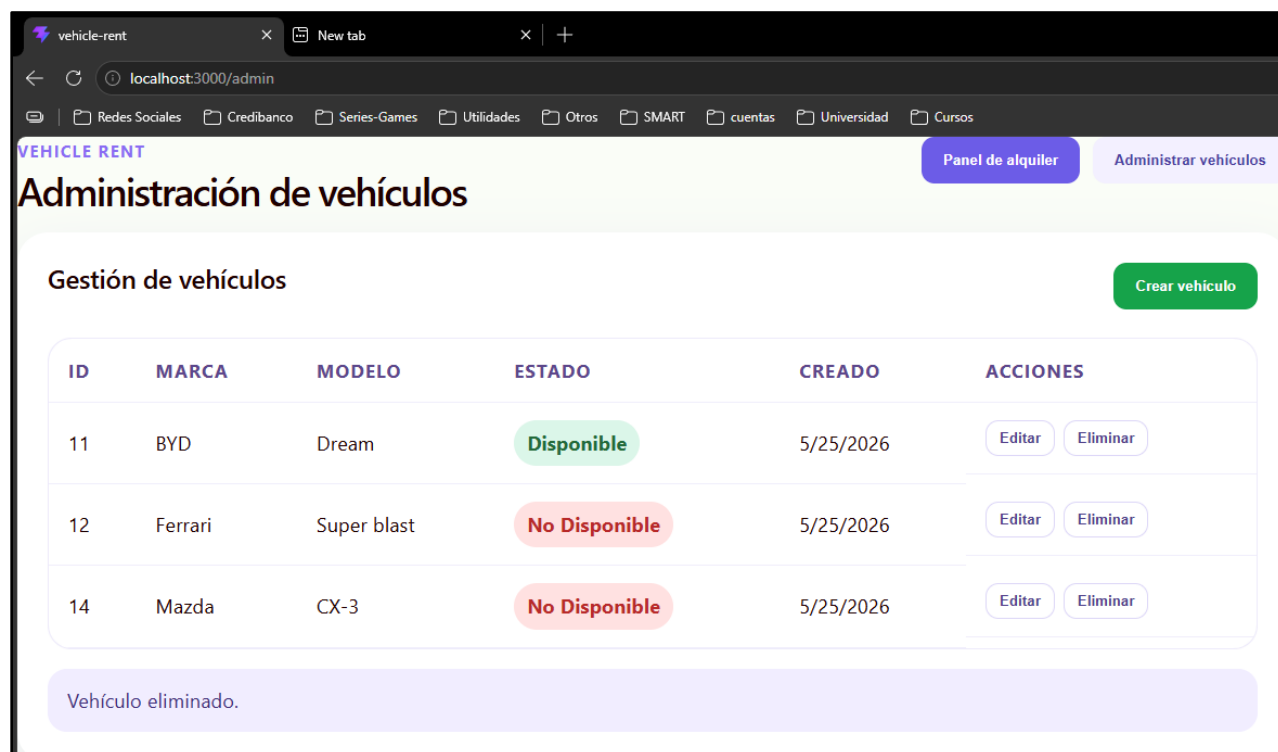


Imagen 6. Panel de administración visualizando la eliminación de vehículos

Panel de alquiler

El panel de alquiler está diseñado de forma que permite visualizar los vehículos disponibles y lo no disponibles, a su vez se expone un botón donde el usuario podrá editar y en la parte superior permite desplazarse entre paneles sin problemas, permitiendo la navegabilidad.

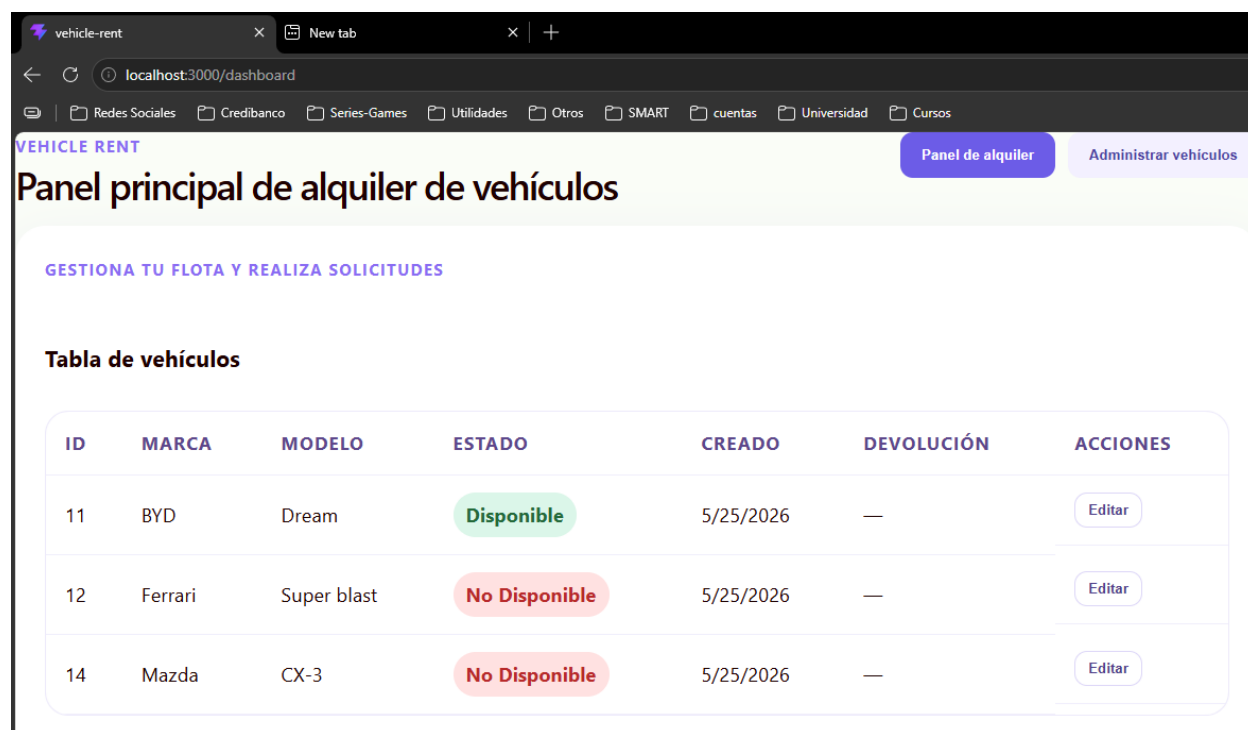


Imagen 7. Panel principal de alquiler de vehículos

Edición de alquiler

Desde el panel principal de alquiler de vehículos, se expone el botón de editar el cual en el mismo panel abre la opción de que el usuario ingrese si el vehículo es disponible o no disponible, a su vez permite colocar la fecha de retorno dentro del rango de 50 días máximos a partir de la fecha actual. Esto con el objetivo de dejar en una pantalla la posibilidad de hacer todo lo necesario y reducir la carga cognitiva del usuario dejando todo disponible. **Ver imagen 8 e imagen 9**

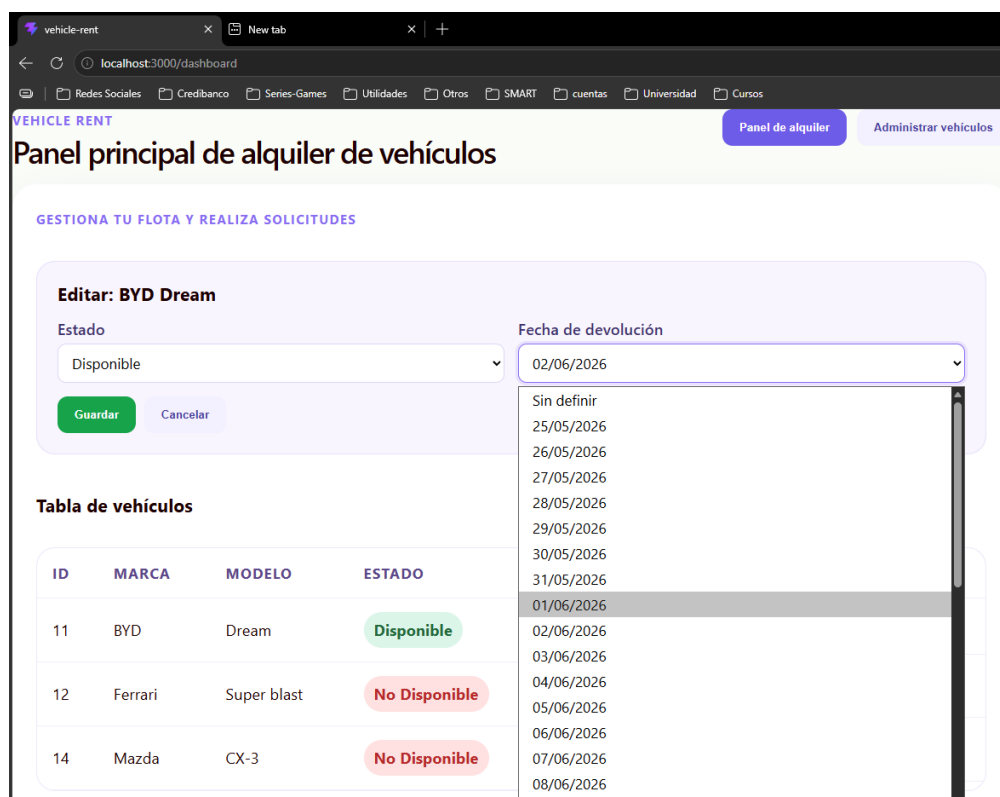


Imagen 8. Opción de cambio de estado y fecha de devolución

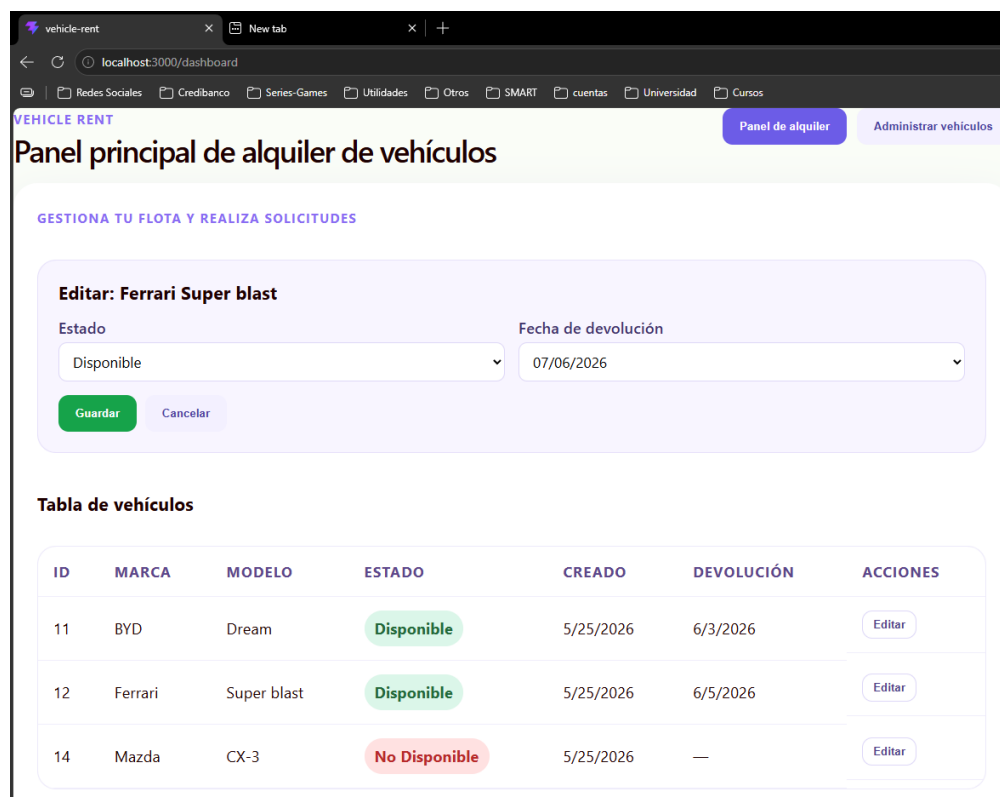


Imagen 9. Actualización de estado y fecha de devolución

Consumo de APIs REST

La integración con el Back-End se realizó a través del módulo `src/services/api.ts`, que encapsula todas las llamadas HTTP hacia dos microservicios independientes.

Arquitectura

Microservicio	URL base	Propósito
vehicle-service	http://localhost:8080	CRUD de vehículos
operation-service	http://localhost:8082	Registro de alquileres

Endpoints consumidos

Método	Endpoint	Función	Descripción
GET	/vehicles	getVehicles()	Obtiene la lista completa de vehículos
DELETE	/vehicles/:id	removeVehicle(id)	Elimina el vehículo seleccionado
PUT	/vehicles/:id	updateVehicleState(id, state)	Actualiza el estado de un vehículo (disponible/no disponible)
POST	/rentals	registerRental(request)	Registra una solicitud de alquiler

Funcionamiento interno

Todas las llamadas pasan por una función genérica `fetchJson<T>()` que:

1. Ejecuta `fetch()` con la URL y opciones recibidas
2. Verifica si la respuesta es exitosa (`response.ok`)
3. Si falla, lanza un error con el código HTTP y el cuerpo del mensaje
4. Si es exitosa, parsea y retorna el JSON tipado

```
async function fetchJson<T>(input: string, init?:  
RequestInit): Promise<T> {  
  
  const response = await fetch(input, init)  
  
  if (!response.ok) {  
  
    const body = await response.text()  
  
    throw new Error(`Error de API: ${response.status}  
${response.statusText} - ${body}`)  
  
  }  
  return response.json()  
}
```

Flujo de alquiler

1. El usuario hace clic en "Alquilar" sobre un vehículo disponible
2. Se llama a `registerRental()` en `operation-service` (POST `/rentals`)
3. Si es exitoso, se llama a `updateVehicleState()` en `vehicle-service` (PATCH `/vehicles/:id/state → "no_disponible"`)
4. El estado se actualiza en la UI sin recargar la página

El consumo de APIs está abstraído en el custom hook `useVehicles`, por lo que ningún componente llama directamente a `fetch()` — toda la comunicación con el Back-End está centralizada en `services/api.ts` y orquestada desde el hook.

Conclusiones

El desarrollo de este proyecto permitió entender mejor cómo funciona React y la manera en que se pueden crear interfaces organizadas usando componentes reutilizables. Aunque algunos temas todavía generan dudas, se logró comprender la lógica general de la estructura del proyecto.

También se aprendió la importancia de separar la lógica del sistema de la parte visual, usando hooks y servicios para mantener el código más limpio y fácil de entender. El uso de React Router ayudó a comprender cómo funcionan las rutas dentro de una aplicación web y cómo es posible navegar entre páginas sin recargar el navegador. Otro aprendizaje importante fue el consumo de APIs REST para conectar el Front-End con el Back-End y así obtener o actualizar información de los vehículos. Esto permitió tener una idea más cercana de cómo funcionan los sistemas reales.

Finalmente, este trabajo sirvió como práctica para reforzar conocimientos de desarrollo web moderno y entender mejor cómo se construye una aplicación completa desde la parte visual hasta la comunicación con servicios externos.

Glosario

Custom Hook

Es una función personalizada de React que permite reutilizar lógica entre diferentes componentes. En este proyecto se usa `useVehicles` para manejar toda la lógica relacionada con los vehículos, como cargar datos, alquilar y eliminar vehículos.

SPA (Single Page Application)

Tipo de aplicación web donde las páginas cambian sin recargar completamente el navegador. Esto hace que la navegación sea más rápida y fluida para el usuario.

React Router

Librería utilizada en React para manejar rutas y navegación entre páginas de la aplicación. Permite cambiar de vistas sin recargar el sitio web.

TypeScript

Superset de JavaScript que agrega tipado de datos. Ayuda a detectar errores antes de ejecutar la aplicación y mejora la organización del código.

API REST

Conjunto de servicios que permiten la comunicación entre el Front-End y el Back-End mediante peticiones HTTP como GET, POST o PATCH.

CRUD

Siglas de Create, Read, Update y Delete. Representa las operaciones básicas que se pueden realizar sobre los datos de una aplicación, como crear, consultar, editar y eliminar vehículos.

Estado (State)

Información dinámica que puede cambiar dentro de un componente React, por ejemplo la lista de vehículos o mensajes mostrados en pantalla. Se maneja usando hooks como useState.

Renderizado

Proceso mediante el cual React actualiza la interfaz gráfica mostrando los cambios realizados en los componentes o en el estado de la aplicación.

Variables de entorno

Valores configurables usados para almacenar información importante como URLs de servicios o configuraciones del sistema sin escribirlas directamente en el código.

Microservicios

Arquitectura donde el sistema se divide en pequeños servicios independientes. En este proyecto existen dos: uno para vehículos y otro para operaciones de alquiler.

Bibliografía

- Learning React, Banks, A. & Porcello, E. (2024). *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media.
- Eloquent JavaScript, Haverbeke, M. (2024). *Eloquent JavaScript* (4th ed.). No Starch Press.
- [React Official Documentation](#). Documentación oficial de React sobre componentes, hooks, renderizado y React Router.
- [TypeScript Documentation](#). Documentación oficial sobre TypeScript y tipado en aplicaciones web.
- [Vite Official Guide](#). Guía oficial de Vite para creación y configuración de proyectos Front-End.
- React Router Documentation. Documentación oficial sobre navegación y manejo de rutas en React.
- Clean Code, Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.